

ONEDRAFT

CAD-AI — Aria Powered

Project Lifecycle & State Machine Architecture

Version 1.0

Document: OneDraft Project Lifecycle & State Machine Architecture Specification

System: OneDraft CAD-AI

Intelligence Layer: Aria

Document Class: Architectural & Implementation Stack Specification

Status: Architectural Specification

Predecessor: Frontend & User Interaction Architecture Specification

Successor: Billing, Subscription & Entitlement Architecture Specification

1. PURPOSE

The Project Lifecycle & State Machine Architecture establishes the formal lifecycle through which a OneDraft project progresses from initial intake through specification, modeling, validation, documentation, review, acceptance and finalization.

The purpose of this subsystem is to provide deterministic project-state management across the complete OneDraft platform.

Project state shall represent the authoritative operational condition of a project at a defined point in its lifecycle.

The lifecycle architecture shall prevent incompatible operations from being executed against projects that are not in an appropriate state.

The state machine shall therefore provide a formal control mechanism connecting user interaction, application services, computational systems, compliance, validation, documentation, review, approval and final project completion.

2. ARCHITECTURAL ROLE

The Project Lifecycle & State Machine subsystem forms the authoritative lifecycle-control layer for OneDraft projects.

It coordinates the progression of project state without replacing the domain-specific services responsible for performing individual operations.

Primary responsibility: Define, enforce and record valid project-state transitions.

Primary inputs: Project commands, requirements, validation results, documentation results, review decisions, acceptance actions and administrative operations.

Primary outputs: Project-state transitions, lifecycle records, transition events, workflow permissions and state-dependent application behavior.

Authority: The backend Project Lifecycle service is authoritative for project lifecycle state.

Determinism: Given an identical project state, authorized command and required transition conditions, the state machine shall produce the same transition result.

3. LIFECYCLE ARCHITECTURE

OneDraft shall represent project lifecycle as a finite state machine.

Each project shall possess a current authoritative lifecycle state.

Transitions between states shall occur only through defined transition operations.

Each transition shall identify:

Previous State: State before the transition.

Target State: State after the transition.

Trigger: Command or event initiating the transition.

Actor: User, system service or authorized process initiating the transition.

Conditions: Requirements that must be satisfied.

Timestamp: Time at which the transition occurred.

Transition Record: Persistent record of the lifecycle change.

4. PROJECT STATE MODEL

The initial canonical project lifecycle shall consist of the following principal states:

INTAKE

SPECIFICATION

MODELING

VALIDATION

DOCUMENTATION

REVIEW

ACCEPTANCE

FINALIZED

Additional operational states may exist where required by the runtime architecture, but they shall not replace the canonical lifecycle states without an explicit architectural revision.

5. INTAKE STATE

INTAKE represents the initial project establishment phase.

A project in INTAKE may contain basic organizational, customer, site and project information.

The project may receive imported information, requirements and initial specifications.

The project is not yet considered sufficiently defined for full computational modeling.

Permitted activities may include project creation, requirement collection, source-data ingestion and preliminary project configuration.

6. SPECIFICATION STATE

SPECIFICATION represents the phase in which project requirements are formally established.

Requirements may include:

Project Requirements

Site Requirements

Program Requirements

Spatial Requirements

Technical Requirements

Regulatory Requirements

Performance Requirements

Documentation Requirements

The specification state establishes the structured requirements against which subsequent modeling and validation operations shall operate.

7. MODELING STATE

MODELING represents the phase in which the computational Project Model is developed.

During MODELING, OneDraft may create and modify:

Buildings

Levels

Grids

Spaces

Elements

Assemblies

Parameters

Constraints

Relationships

Geometry

The Project Model remains subject to applicable validation and lifecycle controls.

8. VALIDATION STATE

VALIDATION represents the phase in which the project is evaluated against applicable requirements.

Validation may include:

Geometry Validation

Constraint Validation

Compliance Validation

Coordination Validation

Model Validation

Documentation Validation

Requirement Validation

Validation results shall be recorded and associated with the relevant project version.

A project shall not progress to subsequent lifecycle states where mandatory validation conditions remain unresolved.

9. DOCUMENTATION STATE

DOCUMENTATION represents the phase in which the validated Project Model is converted into controlled architectural documentation.

Documentation may include:

Plans

Elevations

Sections

Details

Schedules

Annotations

Dimensions

Sheets

Drawing Packages

Documentation shall remain associated with the Project Model version from which it was generated.

10. REVIEW STATE

REVIEW represents the controlled customer, project-team and professional review phase.

Review activities may include:

Drawing Review

Model Review

Redlines

Comments

Revision Requests

Coordination Review

Compliance Review

Documentation Review

Review actions shall be recorded as structured project records.

11. ACCEPTANCE STATE

ACCEPTANCE represents the phase in which required review conditions have been addressed and the project is presented for formal acceptance.

Acceptance shall require satisfaction of the applicable project-specific acceptance criteria.

Authorized users shall be able to approve or reject the project for finalization.

Rejection shall return the project to the appropriate prior lifecycle state according to the defined transition rules.

12. FINALIZED STATE

FINALIZED represents the completed lifecycle state for a defined project release.

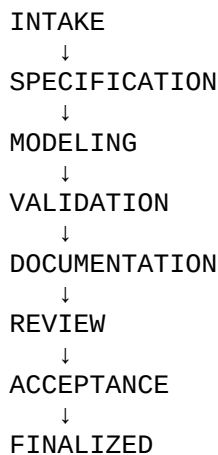
A finalized project shall represent a controlled and versioned project state.

Finalization shall establish the authoritative project version and associated documentation package.

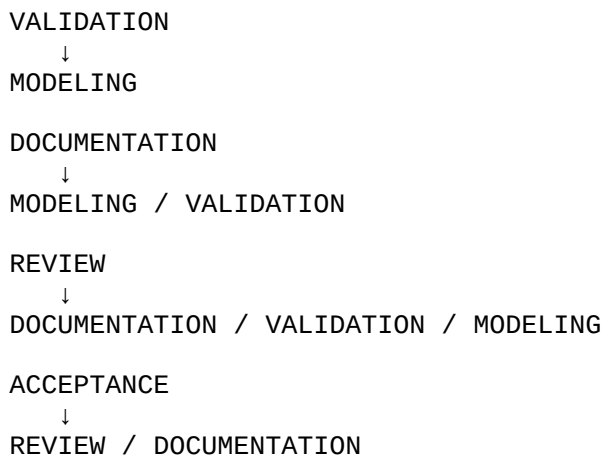
Normal project mutation shall not be permitted against a finalized state without an explicit controlled revision or reopening operation.

13. STATE TRANSITION GRAPH

The canonical lifecycle shall follow the following progression:



Where a project requires corrective work, controlled reverse transitions shall be permitted.



The exact reverse transition shall depend upon the nature of the unresolved condition.

14. TRANSITION AUTHORITY

Every lifecycle transition shall be authorized.

Authorization shall consider:

Actor Identity

Organization Membership

Project Membership

Role

Permission

Project State

Transition Type

Required Conditions

No frontend control shall independently authorize a lifecycle transition.

15. TRANSITION COMMANDS

Lifecycle transitions shall be represented as structured commands.

Representative commands include:

CreateProject

BeginSpecification

BeginModeling

BeginValidation

BeginDocumentation

BeginReview

SubmitForAcceptance

AcceptProject

RejectAcceptance

ReturnToModeling

ReturnToValidation

ReturnToDocumentation

FinalizeProject

ReopenProject

Commands shall be validated before execution.

16. TRANSITION GUARDS

A transition guard shall determine whether a transition is permitted.

Guards may evaluate:

Required Data: Required project information exists.

Requirements: Required specifications are complete.

Model Integrity: Project Model satisfies required structural conditions.

Validation: Mandatory validation has completed.

Documentation: Required drawing outputs exist.

Review: Required review conditions have been addressed.

Acceptance: Authorized acceptance has been recorded.

Security: Actor possesses required authority.

17. TRANSITION VALIDATION

Before a transition is committed, the system shall evaluate all applicable transition conditions.

A failed condition shall prevent the transition.

The system shall return structured information identifying the failed condition.

A lifecycle transition shall never be considered successful merely because the frontend issued the request.

The authoritative lifecycle service shall confirm the resulting state.

18. STATE ENTRY ACTIONS

Each lifecycle state may define entry actions.

Entry actions may include:

INTAKE: Initialize project workspace and intake records.

SPECIFICATION: Activate requirement-management workflows.

MODELING: Activate computational model workflows.

VALIDATION: Initiate or authorize validation workflows.

DOCUMENTATION: Activate documentation generation workflows.

REVIEW: Create review workspace and required review records.

ACCEPTANCE: Establish acceptance package and required approvals.

FINALIZED: Generate final release records and lock the finalized project state.

Entry actions shall execute through the established application and orchestration architecture.

19. STATE EXIT ACTIONS

States may define exit conditions and controlled exit actions.

Exit actions may include validation, record creation, event publication, artifact generation or workflow completion.

Exit actions shall not bypass required domain services.

20. STATE-DEPENDENT OPERATIONS

Project operations shall be evaluated against the current lifecycle state.

For example:

INTAKE: Requirements and source information may be collected.

SPECIFICATION: Requirements may be refined.

MODELING: Model and geometry operations may be performed.

VALIDATION: Validation operations may be executed.

DOCUMENTATION: Drawing-generation operations may be performed.

REVIEW: Review and redline operations may be performed.

ACCEPTANCE: Acceptance decisions may be recorded.

FINALIZED: Controlled release and archival operations may be performed.

State restrictions shall be enforced at the backend.

21. PROJECT VERSIONING

Lifecycle state shall be associated with project versions.

A state transition shall identify the project version against which the transition was evaluated.

This prevents lifecycle decisions from becoming detached from the project information that existed when the decision was made.

22. REVISION INTERACTION

Project revisions and lifecycle states shall operate together.

A revision may cause a project to return to an earlier lifecycle state when the change materially affects requirements, geometry, compliance, validation or documentation.

The system shall determine the required lifecycle response according to defined transition rules.

Minor documentation changes may not require a complete lifecycle reset where project policy permits.

23. CHANGE IMPACT

The lifecycle engine shall support change-impact evaluation.

Changes may affect:

Requirements

Geometry

Spaces

Assemblies

Compliance

Validation

Drawings

Schedules

Review Results

The system shall identify which lifecycle conditions may have become invalid as a result of a change.

24. VALIDATION GATES

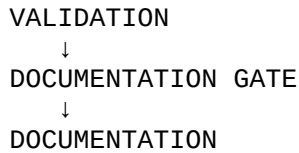
Certain transitions shall require validation gates.

A validation gate shall identify the minimum conditions required for progression.

Example:

```
MODELING
  ↓
VALIDATION GATE
  ↓
VALIDATION
```

and:



Validation gates shall be deterministic and versioned.

25. DOCUMENTATION GATES

Documentation progression shall require the necessary project information and model conditions.

The system shall verify that required documentation inputs exist before entering or completing DOCUMENTATION.

Generated drawings shall remain linked to the applicable Project Model version.

26. REVIEW GATES

A project shall enter REVIEW only when the required documentation package has been generated and applicable prerequisite conditions have been satisfied.

The review gate shall verify:

Required Sheets: Present.

Required Drawings: Present.

Required Validation: Complete.

Required Metadata: Present.

Required Artifacts: Registered.

27. ACCEPTANCE GATES

Acceptance shall require satisfaction of defined review conditions.

The acceptance gate may verify:

Mandatory Redlines: Resolved.

Required Reviews: Complete.

Validation: No blocking conditions.

Documentation: Complete.

Required Approvals: Obtained.

The exact acceptance criteria shall be configurable through the administrative and project configuration architecture.

28. FINALIZATION GATE

Finalization shall be a controlled operation.

The finalization gate shall verify all mandatory project conditions.

Finalization may require:

Validation Completion

Documentation Completion

Review Completion

Acceptance

Artifact Registration

Revision Integrity

Provenance Integrity

Required Approvals

Only an authorized actor or authorized system workflow may finalize a project.

29. REOPENING

A finalized project may be reopened through a controlled operation.

Reopening shall create an explicit lifecycle record identifying:

Previous Finalized Version

Reason for Reopening

Actor

Time

New Working Version

The finalized version shall remain preserved.

Reopening shall not erase the historical finalized state.

30. PROJECT SUSPENSION

OneDraft shall support controlled project suspension where operational requirements require it.

Suspension shall preserve the underlying project state while preventing selected project operations.

Suspension may occur due to administrative conditions, customer direction, operational requirements or other authorized conditions.

Suspension shall not destroy project data or lifecycle history.

31. PROJECT CANCELLATION

Project cancellation shall be a controlled lifecycle operation.

A cancelled project shall preserve its historical records according to retention requirements.

Cancellation shall not be equivalent to deletion.

The system shall distinguish project cancellation from data deletion.

32. LIFECYCLE EVENTS

Every successful state transition shall publish a lifecycle event through the OneDraft event architecture.

Representative events include:

ProjectEnteredSpecification

ProjectEnteredModeling

ProjectEnteredValidation

ProjectEnteredDocumentation

ProjectEnteredReview

ProjectSubmittedForAcceptance

ProjectAccepted

ProjectFinalized

ProjectReopened

These events shall permit downstream services to respond without tightly coupling them to the lifecycle engine.

33. EVENT CONSISTENCY

Lifecycle events shall be generated only from authoritative state transitions.

The system shall prevent duplicate lifecycle events where possible.

Event processing shall support the established asynchronous job and event-bus architecture.

Lifecycle state shall remain authoritative even if downstream event processing is delayed.

34. ARIA INTEGRATION

Aria shall be aware of the current project lifecycle state.

Aria shall use lifecycle state when interpreting project commands and determining available actions.

For example, Aria shall not propose finalization as though it were available when the project remains in MODELING or unresolved VALIDATION conditions.

Aria may explain lifecycle requirements and recommend appropriate next actions.

Aria shall not independently override lifecycle guards.

35. AI-INITIATED TRANSITIONS

Aria may propose lifecycle transitions where the user's instruction and project context permit.

AI-generated transition proposals shall be converted into structured commands.

The command shall pass through authorization, transition validation and applicable confirmation requirements.

Aria shall never directly modify the authoritative lifecycle state.

36. FRONTEND INTEGRATION

The frontend shall display the current project lifecycle state prominently within the active project workspace.

The interface shall adapt available controls according to lifecycle state.

Users shall be able to inspect:

Current State

Previous State

Available Transitions

Blocked Transitions

Required Conditions

Lifecycle History

The frontend shall obtain this information from the authoritative lifecycle service.

37. LIFECYCLE HISTORY

OneDraft shall maintain a persistent lifecycle history.

The history shall record all successful transitions and relevant rejected transition attempts where required for auditability.

Lifecycle history shall include:

Transition Identifier

Project Identifier

Previous State

New State

Trigger

Actor

Timestamp

Conditions

Result

38. AUDITABILITY

Lifecycle operations shall be fully auditable.

The lifecycle audit architecture shall integrate with identity, access control, change records, observability and data provenance.

A historical project state shall remain reconstructable from lifecycle and revision records to the extent required by the platform's retention policies.

39. SECURITY

Lifecycle operations shall be protected by the OneDraft identity and authorization architecture.

Privileged transitions such as acceptance, finalization and reopening shall require appropriate authorization.

The frontend shall not be considered a security boundary.

All lifecycle permissions shall be enforced server-side.

40. MULTI-TENANT ISOLATION

Lifecycle state shall remain isolated within the applicable organization and project tenant boundary.

A user shall not be able to transition a project belonging to another tenant.

Lifecycle events and history shall retain tenant context.

Cross-tenant lifecycle operations shall not be permitted unless explicitly defined through an authorized administrative architecture.

41. OBSERVABILITY

Lifecycle operations shall produce structured telemetry.

Telemetry shall include:

Project Identifier

Current State

Target State

Transition

Actor

Duration

Result

Failure Reason

Operational monitoring shall identify repeated transition failures, stalled workflows and abnormal lifecycle behavior.

42. TESTING

The lifecycle engine shall be subject to deterministic state-machine testing.

Testing shall include:

Valid Transitions

Invalid Transitions

Authorization Failures

Missing Preconditions

Validation Failures

Concurrent Changes

Reopening

Cancellation

Suspension

Finalization

Event Publication

Recovery

Every canonical transition shall have automated test coverage.

43. CONCURRENCY

The lifecycle service shall protect against conflicting concurrent transitions.

A transition shall be evaluated against the current authoritative state.

Optimistic concurrency or equivalent mechanisms shall prevent two incompatible transitions from simultaneously becoming authoritative.

Conflicting operations shall return structured conflict results.

44. FAILURE RECOVERY

Lifecycle operations shall be recoverable following application or worker failures.

A partially executed transition shall not leave the project in an undefined state.

State mutation and lifecycle-event publication shall follow the established transaction and event-consistency architecture.

Recovery procedures shall preserve lifecycle history.

45. CONFIGURATION

Lifecycle rules shall be configurable where project or product requirements require controlled variation.

Configuration may define:

Required Reviewers

Validation Gates

Acceptance Requirements

Allowed Transitions

Finalization Requirements

Reopening Rules

Configuration shall be versioned and governed by the Product Configuration & Administrative Control Plane.

46. API ARCHITECTURE

The lifecycle API shall expose controlled resources and operations.

Representative endpoints shall include:

GET /projects/{id}/lifecycle

GET /projects/{id}/lifecycle/history

GET /projects/{id}/lifecycle/transitions

POST /projects/{id}/lifecycle/transition

POST /projects/{id}/lifecycle/validate-transition

POST /projects/{id}/lifecycle/reopen

The final contract shall remain governed by the established OneDraft API and OpenAPI architecture.

47. DATA MODEL REQUIREMENTS

The lifecycle architecture shall support structured lifecycle entities.

Representative entities include:

ProjectLifecycle

LifecycleState

LifecycleTransition

TransitionGuard

TransitionCondition

LifecycleEvent

LifecycleHistoryRecord

LifecycleGate

LifecycleConfiguration

ProjectSuspension

ProjectCancellation

These structures shall integrate with Project, ModelVersion, Revision, ChangeRecord, AriaCommand, ValidationResult, Review and Acceptance structures.

48. ARCHITECTURAL INVARIANTS

The following invariants shall govern the Project Lifecycle & State Machine Architecture.

Single Authoritative State: Each project shall have one authoritative current lifecycle state.

Controlled Transition: State changes shall occur only through defined transition operations.

No Invalid Transition: Transitions failing required conditions shall not become authoritative.

Backend Authority: Frontend state shall never override lifecycle state.

Authorization: Every transition shall be subject to authorization.

Revision Integrity: Lifecycle state shall remain associated with the applicable project version.

Historical Preservation: Prior lifecycle states shall remain historically traceable.

No Silent Finalization: Finalization shall always be an explicit controlled operation.

No Silent Reopening: Reopening a finalized project shall create an explicit historical record.

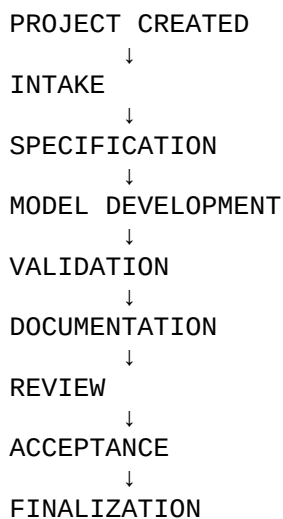
AI Separation: Aria may propose lifecycle actions but shall not independently override lifecycle controls.

Event Integrity: Lifecycle events shall originate from authoritative transitions.

Tenant Isolation: Lifecycle state shall remain within its authorized organizational and project boundary.

49. END-TO-END LIFECYCLE

The canonical OneDraft project lifecycle is therefore:



↓
CONTROLLED FINAL PROJECT RELEASE

Where corrections are required, the lifecycle may return to the appropriate prior state without destroying historical versions.

The lifecycle architecture therefore provides a controlled progression from initial project information through structured requirements, computational modeling, deterministic validation, documentation generation, review, acceptance and final project release.

50. COMPLETION STATUS

The Project Lifecycle & State Machine Architecture establishes the formal operational state model governing OneDraft projects.

The subsystem connects project intake, specification, computational modeling, validation, documentation, review, acceptance and finalization into one controlled lifecycle.

It establishes deterministic state transitions, transition guards, validation gates, documentation gates, review gates, acceptance requirements, finalization controls, reopening, suspension, cancellation, lifecycle events, authorization, auditability, concurrency protection and Aria integration.

The lifecycle architecture therefore provides the state-control mechanism required for OneDraft to operate as a coherent end-to-end project platform.

Status: ARCHITECTURALLY DEFINED

Next Document: Billing, Subscription & Entitlement Architecture Specification

Overall Architecture: OneDraft End-to-End System Integration